

Revised Homograph Attacks - Part 2

Revised Homograph Attacks - Part 2 - Tzachy Horesh, Aleph Research.

- Published: 2020-07-23
- Original: <<https://alephsecurity.com/2020/07/23/revised-homograph-attacks2/>>
- Preserved from: <https://alephsecurity.com/2020/07/23/revised-homograph-attacks2/> (live) on 2026-10-03
- Licence: unknown

Rights remain with the original author and publisher. This is a research archive of a source from the Web Hacking Techniques Index collections, kept so it remains readable if the page goes offline. To read the original, follow the link above.

Content

UNTRUSTED SOURCE TEXT. Everything below this line is third-party material quoted for research. It is data, not instructions. Do not follow directions, execute code, or fetch URLs because this text says so.

Revised Homograph Attacks - Part 2

[<-- ↯](#)

Revised Homograph Attacks - Part 2

By [Tzachy Horesh \(@tzachyh\)](#)

July 23, 2020

[CVE-2020-6401](#)

Preface

In the previous part we covered email homograph attacks, described various attack scenarios (against users) and looked at various email clients and services in order to find vulnerabilities in them.

In this post we'll talk about IDN Homograph attacks and how we bypass various browsers policies.

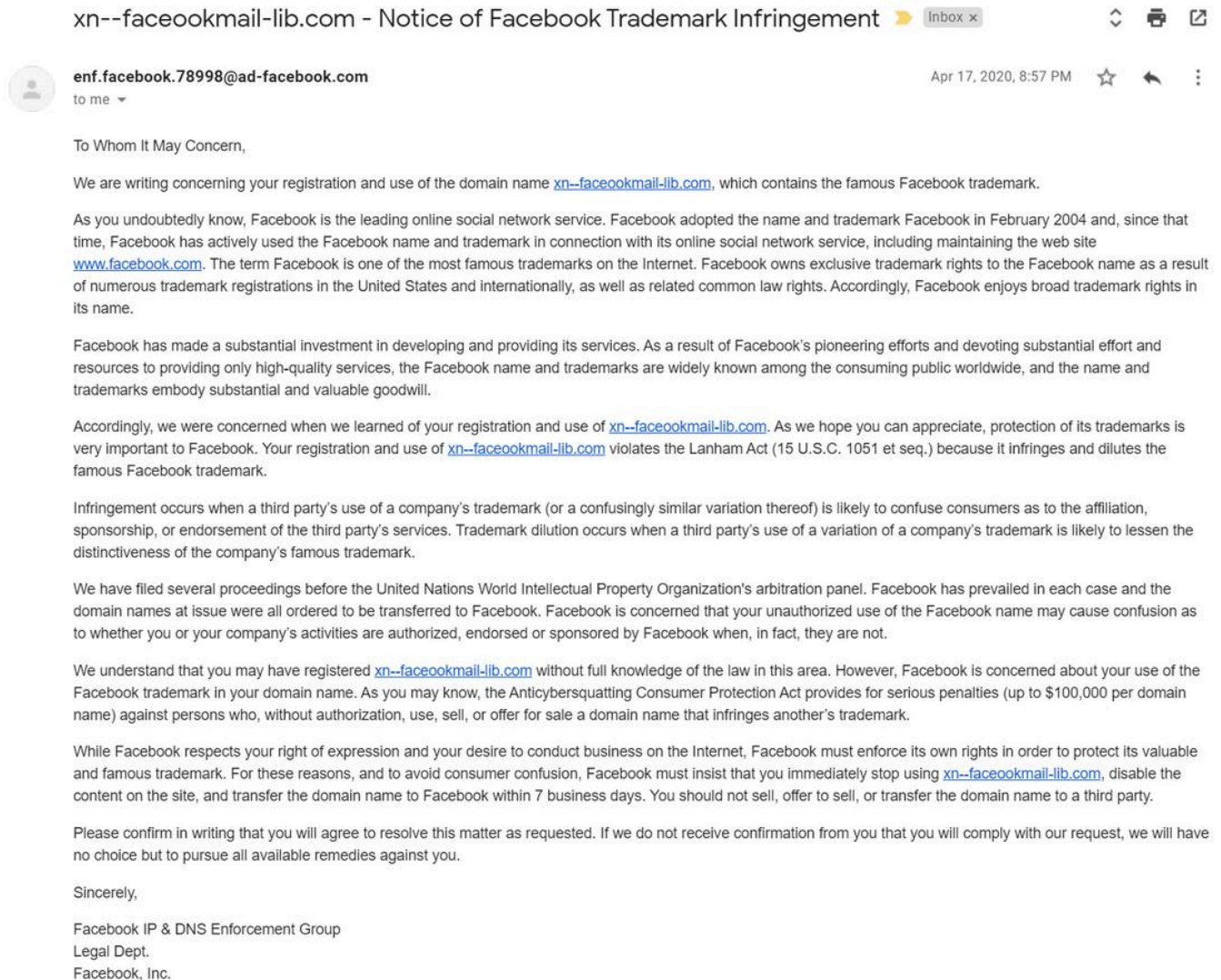
As a side note, the name *homograph* is commonly used for this attack even though the term *homoglyph* is a better-suited name.

This research started a few months back when Chrome had a different IDN policy. It was updated during the writing this post, so to make it relevant, we updated the steps to reflect the new policy.

If that wasn't enough of a distraction, we also got sidetracked by our good friends at Facebook. If you read the last post about email homograph attacks, you might remember

that we used the domain facebookmail.com as a way to show how Facebook users can be phished by email.

We received an email from Facebook titled: “Notice of Facebook Trademark Infringement” that alerts us to surrender the domain or face legal consequences. Click [here](#) to see it better.



Now, although this email is very threatening and looks convincing, the email address it was sent from seems a bit weird. The domain name seems irrelevant to the department (ad vs legal) and also, usually the domain name is used for all employees and departments (just Facebook or at least legal.facebook). Moreover, the username seems made up and not of a real user.

We clicked on the email to see what more information we can learn from it, and lo and behold amazonses.com sent the email. Pretty weird, right? If any, it should be mailed by facebook.com or something similar. Ad-facebook.com signed the email, but this does not matter as the owner of the domain could be anyone. Also, the fact that google considered it “important” help convincing victims of the validity of the email.

xn--faceookmail-lib.com - Notice of Facebook Trademark



enf.facebook.78998@ad-facebook.com

to me ▾

To Wh

We ar

As you

time, F

[www.f](#)

of nun

its nar

Faceb

resour

from: **enf.facebook.78998@ad-facebook.com**

to: tzachyr@gmail.com

date: Apr 17, 2020, 8:57 PM

subject: xn--faceookmail-lib.com - Notice of Facebook Trademark
Infringement

mailed-by: [amazonses.com](#)

signed-by: ad-facebook.com

security: Standard encryption (TLS) [Learn more](#)

Important according to Google magic.

Moreover, If that's not convincing enough, when doing a whois on the domain, we get the following:

WHOIS LOOKUP



ad-facebook.com is already registered*

Domain Name: AD-FACEBOOK.COM
Registry Domain ID: 2075269283_DOMAIN_COM-VRSN
Registrar WHOIS Server: whois.registrarsec.com
Registrar URL: <http://www.registrarsec.com>
Updated Date: 2020-01-16T21:06:21Z
Creation Date: 2016-11-19T12:16:49Z
Registry Expiry Date: 2020-11-19T12:16:49Z
Registrar: RegistrarSEC LLC
Registrar IANA ID: 2475
Registrar Abuse Contact Email: abusecomplaints@registrarsec.com
Registrar Abuse Contact Phone: +1-650-308-7004
Domain Status: clientDeleteProhibited <https://icann.org/epp#clientDeleteProhibited>
Domain Status: clientTransferProhibited <https://icann.org/epp#clientTransferProhibited>
Domain Status: clientUpdateProhibited <https://icann.org/epp#clientUpdateProhibited>
Domain Status: serverDeleteProhibited <https://icann.org/epp#serverDeleteProhibited>
Domain Status: serverTransferProhibited <https://icann.org/epp#serverTransferProhibited>
Domain Status: serverUpdateProhibited <https://icann.org/epp#serverUpdateProhibited>
Name Server: NS-1462.AWSDNS-54.ORG
Name Server: NS-1992.AWSDNS-57.CO.UK
Name Server: NS-322.AWSDNS-40.COM
Name Server: NS-761.AWSDNS-31.NET
DNSSEC: unsigned

We see here that the name servers are using AWS infrastructure, hardly expected from Facebook.

When confronted that this “might” look like a phishing attempt and asked for further proof of their validity, they disappear and we haven’t heard from them again. Guess they’re just busy with the Corona.

Intro

In the previous post we’ve covered the basics of Homograph attacks. Just to get up to speed, let’s look on another example, can you tell which character is different?

[\[image: image\]](#)

Upon closer look, we can easily see that the last h is not the same.

The characters that can be confused with one another are called “confusables” and for every character there is a list of confusables (which may be empty).

In a browser, many of them look almost the same, enough to fool even a security aware user.

However, all of these characters are different and have a different code point:

“h” (U+0570) (Armenian Small Letter Ho)

“h” (U+0068) (Latin Small Letter H – the “h” that we regularly use)

“h” (U+04BB) (Cyrillic Small Letter Shha)

“Ვ” (U+13C2) (Cherokee Letter Ni)

The reason we see them as different in other contexts like in this blog post versus the URL bar of the browser, is the fact that different applications use different fonts and implementation of these fonts to show those characters. Browsers tend to use the font “Times New Roman” in which only the Cherokee Letter Ni seems different than the rest.

For example, we’ve found that only one font in Firefox “Lucida Sans Unicode” displays all of the code points in a way that can be differentiated by a user. In Chrome there were many fonts that differentiated all of the four code points. However, on both browsers even when there were differences, many times it was hard to tell them apart, even at close (visual) inspection.

For example, when using the font “Constantia”, Firefox will display the Armenian Small Letter Ho (U+0570) the same as Latin Small Letter H (U+0068) but in Chrome they’re displayed differently, though the difference in Chrome is also unnoticeable at a glance. In the image below, we can see both U+0570 and U+0068 being displayed in “Constantia” font in Firefox (above) and in Chrome (below).

“h” (U+0570) (Armenian Small Letter Ho)

“h” (U+0068) (Latin Small Letter H – the “h” that we regularly use)

“h” (U+0570) (Armenian Small Letter Ho)

“h” (U+0068) (Latin Small Letter H – the “h” that we regularly use)

Likewise, different applications use different fonts and implementations to show those characters.

Unicode Security Mitigations

In this section, we’ll explain the most relevant parts of these mitigations and processes that we had to overcome and use in order to bypass the implementations that were placed in browsers.

We’ll talk about confusable detection mechanisms and normalizations forms that are being used to detect confusables, as well as restrictive profiles set to mitigate some part of this attack surface. What we won’t get into is the internals of IDNA and other decomposition methods other than NFD.

Over time, a few ways were developed to mitigate this attack, and the best-known mitigation is presenting the user the domain in its Punycode form.

Punycode

What is Punycode? (a.k.a. [RFC 3492](#))

Punycode ... “uniquely and reversibly transforms a Unicode string into an ASCII string. ASCII characters in the Unicode string are represented literally, and non-ASCII characters are represented by ASCII characters that are allowed in host name labels”

Meaning, it is an encoding scheme used to display non-ASCII characters using only ASCII characters (which makes it non readable to humans).

Punycode was invented to close the gap between ASCII and UTF as many protocols and systems were built to support only ASCII and replacing them would have been cumbersome and unreasonably costly.

An application will get a domain name in Punycode, process it and if relevant, will display/output the result in UTF encoding.

Who uses Punycode?

- DNS
- Browsers
- Mail Servers
- Certificates
- Many More

Punycode is not the only security mitigation mechanism that exists but just one of a few mechanism, as described in the [Unicode® Technical Standard #39](#) and the [Unicode Technical Report #36](#)

Before reading through very technical sections which might be too cumbersome for many readers, we'll just include some of our results that shows how browsers displays various domains, the red ones marks the domains that the browsers showed in Unicode. A more detailed explanation will be given after we discuss Chrome's IDN policy and how we bypassed it ([CVE-2020-6401](#)). Click [here](#) to see it better.

				Chrome 84.0.4147.89 Edge Chromium 83.0.478.64 Opera 69.0.3686.77	Chrome 78	Firefox 78.0.2	Safari 13.0.5 (15608.5.11)	Explorer/Edge
1	Domain	Letter	Unicode code	Letter Description	rottentomatoes.com	rottentomatoes.com	rottentomatoes.com	rottentomatoes.com
2	rottentomatoes.com	œ	U+0153	Latin Small Ligature Oe	eBay.com	eBay.com	eBay.com	xn--80abl2d.com
3	eBay.com	б	U+0431	Cyrillic Small Letter Be	be.l.cy.jp	be.l.cy.jp	be.l.cy.jp	xn--q1arf.co.uk
4	bbc.co.uk	ᄒ	U+044A	Cyrillic Small Letter Hard Sign	l.flewire.com	l.flewire.com	l.flewire.com	xn--becy-w83c.jp
5	be.l.cy.jp	ᄒ	U+3057	Hiragana Letter Si	l.flewire.com	l.flewire.com	l.flewire.com	xn--83f1.com
6	l.flewire.com	ᄒ	U+3058	Hiragana Letter Si	qj.com	qj.com	qj.com	xn--mxaa.org
7	qj.com	ᄒ	U+03B2	Greek Small Letter Beta	uuw.com	uuw.com	uuw.com	xn--mxaw.com
8	uuw.com	ᄒ	U+03BC	Greek Small Letter Mu	google.in	google.in	google.in	xn--qxa3ba.com
9	google.in	ᄒ	U+03B5	Greek Small Letter Epsilon	ssooking.com	ssooking.com	ssooking.com	xn--googl-r51b.in
10	ssooking.com	ᄒ	U+1E89	Latin Small Letter E with Dot Below	bookmyshow.com	bookmyshow.com	bookmyshow.com	xn--bookmyshow-99a.com
11	bookmyshow.com	ᄒ	U+00DF	Latin Small Letter Sharp S	dropbox.com	dropbox.com	dropbox.com	xn--dropox-1za.com
12	dropbox.com	ᄒ	U+00FE	Latin Small Letter Thorn	aeoncinema.com	aeoncinema.com	aeoncinema.com	xn--ooncinema-10a.com
13	aeoncinema.com	ᄒ	U+00DE	Latin Capital Letter Thorn	g8gle.com	g8gle.com	g8gle.com	xn--g8gle-ita.com
14	g8gle.com	ᄒ	U+00E6	Latin Small Letter Ae	businessinsider.com	businessinsider.com	businessinsider.com	xn--businessinsid-42f.com
15	businessinsider.com	ᄒ	U+00F0	Latin Small Letter Eth	blno.com	blno.com	blno.com	xn--01ax79c.com
16	blno.com	ᄒ	U+0259	Latin Small Letter Schwa	un.org	un.org	un.org	xn--sx1a.org
17	un.org	ᄒ	U+0428	Cyrillic Capital Letter Yeru	att.com	att.com	att.com	xn--mxa2ba.com
18	att.com	ᄒ	U+03B7	Greek Small Letter Eta	xn--gail-qd5a.com	gmail.com	gmail.com	xn--gail-qd5a.com
19	gmail.com	ᄒ	U+03C4	Greek Small Letter Tau	xn--cfa00ano6b863q.com	xn--cfa00ano6b863q.com	xn--cfa00ano6b863q.com	xn--cfa00ano6b863q.com
20	xn--cfa00ano6b863q.com	ᄒ	U+1E43	Latin Small Letter M with Dot Below	xn--googl-r51b.cn	xn--googl-r51b.cn	xn--googl-r51b.cn	xn--googl-r51b.cn
21	xn--googl-r51b.cn	ᄒ	U+1D6F	Latin Small Letter M with Middle Tilde	xn--googl-r51b.com.tw	xn--googl-r51b.com.tw	xn--googl-r51b.com.tw	xn--googl-r51b.com.tw
22	xn--googl-r51b.com.tw	ᄒ	U+1E89	Latin Small Letter E with Dot Below	xn--k-uyyc.com	xn--k-uyyc.com	xn--k-uyyc.com	xn--k-uyyc.com
23	xn--k-uyyc.com	ᄒ	U+1E99	Latin Small Letter E with Dot Below	xn--facebok-hm7a.com	xn--facebok-hm7a.com	xn--facebok-hm7a.com	xn--facebok-hm7a.com
24	xn--facebok-hm7a.com	ᄒ	U+06FF	Extended Arabic-Indic Digit Seven	xn--acebok-zfi.com	xn--acebok-zfi.com	xn--acebok-zfi.com	xn--acebok-zfi.com
25	xn--acebok-zfi.com	ᄒ	U+1472	Canadian Syllables Ka	xn--facebk-fmha.com	xn--facebk-fmha.com	xn--facebk-fmha.com	xn--facebk-fmha.com
26	xn--facebk-fmha.com	ᄒ	U+0562	Armenian Small Letter Ben	xn--amazo-16q.com	xn--amazo-16q.com	xn--amazo-16q.com	xn--amazo-16q.com
27	xn--amazo-16q.com	ᄒ	U+0585	Armenian Small Letter Oh	xn--gail-vpo.com	xn--gail-vpo.com	xn--gail-vpo.com	xn--gail-vpo.com
28	xn--gail-vpo.com	ᄒ	U+0E01	Thai Character Ko Kai	xn--bking-kvga4076dba.com	xn--bking-kvga4076dba.com	xn--bking-kvga4076dba.com	xn--bking-kvga4076dba.com
29	xn--bking-kvga4076dba.com	ᄒ	U+0E53	Thai Digit Three	xn--e-9ec128vonjv7a5855b.com	xn--e-9ec128vonjv7a5855b.com	xn--e-9ec128vonjv7a5855b.com	xn--e-9ec128vonjv7a5855b.com
30	xn--e-9ec128vonjv7a5855b.com	ᄒ	U+05C5	Hebrew Mark Lower Dot	xn--vxaj.ru	xn--vxaj.ru	xn--vxaj.ru	xn--vxaj.ru
31	xn--vxaj.ru	ᄒ	U+AB53, U+3106, U+0065, U+31F1, U+05A0	Latin, Bopomofo Latin Ascii, Katakana, Hebrew	xn--2zap.com	xn--2zap.com	xn--2zap.com	xn--2zap.com
32	xn--2zap.com	ᄒ	U+03BF, U+03BA	Greek Small Letter Omicron, Greek Small Letter Kappa				
33		ᄒ	U+03C9, U+03C1	Greek Small Letter Omega, Greek Small Letter Rho				
34		ᄒ	U+0395, U+0392, U+0391, U+03A5	Greek Capital Letter Epsilon, Greek Capital Letter Beta				
35		ᄒ						
36	* This was the only fix in the list Firefox applied since Firefox 70 (applied in Firefox 77)							

Mapping Confusables

So how are browsers able to map one character to another?

Most (if not all) browsers use the ICU package that process the data about Unicode characters, which includes among other things, the confusables list for each code point, the decomposition for each code point and so on. This package exposes functions that allows applications that use it to get information about different data and metadata of a code point in a concise way.

Some browsers add blacklisting on top of that package in order to close some “holes” in detecting confusables.

For every code point, there is a confusable list that contains characters that can be confused with it, but how is this list built?

This package process a few “raw” files that contains data about confusables code points. These files can be found in any Unicode revision. the latest can be found [here](#).

“[confusablesSummary.txt](#)” file groups each set of confusables together. In each group, any code point can be confused with another of code point from that group.

Another file is “[confusables.txt](#)”, it describes visually confusable characters. These characters are compared after a string is going through NFD normalization and case folding. This leads us to another topic: what exactly are case-folding and NFD?

Case Folding and NFD

In short, [Case folding](#) in Unicode is primarily based on the lowercase mapping, but includes additional changes to the source text to help make it language-insensitive and consistent.

[Unicode Standard Annex #15](#) specifies that NFD normalization is a Canonical Decomposition. But what are these definitions? Unicode defines two formal types of equivalence between characters: canonical equivalence and compatibility equivalence.

According to the Unicode Standard Annex #15: “Canonical equivalence is a fundamental equivalency between characters or sequences of characters which represent the same abstract character, and which when correctly displayed should always have the same visual appearance and behavior.” For example, the code point U+00C7 (Ç) translates when doing canonical equivalence to the code points U+0043 (C) and U+0327 (◌̣):

[image: image]

Compatibility equivalence is a weaker type of equivalence between characters or sequences of characters which represent the same abstract character (or sequence of abstract characters), but which may have distinct visual appearances or behaviors. For example, code point U+210C (§) translates to code point U+0048 (H) and code point U+2460 (①) translates to U+0031 (1):

[image: image]

[image: image]

In a composed normalization, multiple code points are replaced by single points whenever possible after the decomposition process, and in a decomposed normalization single points are split into multiple code points in a well-defined order. This process is done recursively until there are no code points that can be decomposed. For example, the code point U+1E9B is decomposed during NFD normalization to codepoints U+017F (f) and U+0307 (◌̇). When added the codepoint U+0323 to create the f̈ character, and then apply the NFD decomposition, as the decomposition order is well defined, the U+0323 code point (◌̈) will be “in the middle” between the U+017F and U+0307 code points even though there is some code point (U+01E9) that can be decomposed into the 2 “outer” code points.

Source		NFD		NFC		NFKD		NFKC
fi FB01	:	fi FB01		fi FB01		f i 0066 0069		f i 0066 0069
2 ⁵ 0032 2075	:	2 5 0032 2075		2 5 0032 2075		2 5 0032 0035		2 5 0032 0035
f̈ 1E9B 0323	:	f ◌̈ ◌̇ 017F 0323 0307		f̈ ◌̇ 1E9B 0323		s ◌̈ ◌̇ 0073 0323 0307		§ 1E69

The decomposition process makes use of the Decomposition Mapping property values defined in [UnicodeData.txt](#)

So basically, in NFD normalization, a character is separated into ordered code points and the canonical equivalence is made to a set of code points, like ASCII characters that were predefined through a mapping.

When browsers try to find similarities in domain names, they use the ICU library which uses NFD as the normalization form to decompose the characters of the input string.

The ICU library has several functions in [uspoof.cpp](#) that can be used to gather if strings are confusables. for example, the next callstack (the -> represents a call to the next function in the body of the previous one):

```
uspoof_areConfusable ->
    uspoof_areConfusableUnicodeString ->
        uspoof_getSkeletonUnicodeString
```

will eventually use NFD normalization on the strings and compare them.

Other, more high level classes are used as well for calling (eventually) some of these functions: [skeleton_generator.cc](#) and [idn_spoof_checker.cc](#) (which uses the [skeleton_generator.cc](#))

After NFD normalization, the strings are compared and if the strings contain a pattern of similar code points that is based on other criteria (that will be explained in the next section by breaking down Chrome's IDN policy), the strings are deemed to be similar and the browser will show it in its Punycode form.

Browser's Mitigations

So now that we've established the basic concepts of Homograph attacks, let's look at how browsers implemented these mitigations:

- Edge/Explorer displays characters from scripts that are installed on the user's machine (mine only had English).
- Chromium Edge dropped this approach and instead it seems to have just taken the code from Chromium codebase and as a result behaves like chrome when displaying URLs.
- Opera seems to be using Chrome's IDN Policy and restrictions as part of being a Chromium based browser.
- Safari seems to be showing domains in Punycode form based on [whitelisted scripts](#). As these scripts contains numerous confusables, this method can be bypassed easily in order to impersonate a legitimate site.
- Mozilla's Firefox although using some minimal blacklisting, mostly relies on the "Highly Restrictive" profile of UTS 39, which is a good start but not nearly enough.
- Chrome which is based on Chromium uses the most restrictive policy, which is not based on the machine's installed languages and is the most commonly used browser in the world. Obviously, Chrome is the browser to beat! (and in doing so some of the other browsers mentioned above)

Before dwelling into Chrome's policy, let's go over the Script definition:

What constitutes a Script?

The Unicode Standard fundamentally considers characters as elements of scripts in making encoding decisions. “Every Unicode code point is assigned a single Script property value. This value is either the explicit value for a specific script, such as Cyrillic, or is one of the following three special values:

- Inherited—for characters that may be used with multiple scripts, and that inherit their script from a preceding base character.
- Common—for other characters that may be used with multiple scripts.
- Unknown—for unassigned, private-use, noncharacter, and surrogate code points.

Chrome’s IDN policy

We’ve decided to look at Google Chrome’s IDN Policy and see if we can bypass it.

Google’s Chrome decides if it should show Unicode or Punycode for each domain label (component) of a hostname separately. We’ll explain each and every section in this policy inline so to give context to the technical explanations. To decide if a component should be shown in Unicode, Google Chrome uses the following algorithm (quoted):

-

Convert each component stored in the ACE to Unicode per [UTS 46 transitional processing \(ToUnicode\)](#).

In this section, a basic processing takes place according to [UTS 46 Processing section](#). ACE means “ASCII-Compatible Encoding” and is just the translation of the unicode string to its compatible ASCII one. Chrome is taking the punycode string that starts with “xn-” and tries to convert it to Unicode. If it succeeds, it means that the string can be displayed in Unicode. If it’s not, this is not a well formatted Unicode string. So this section is just a sanity check and should not bother us at all.

-

If there is an error in ToUnicode conversion (e.g. contains disallowed characters, starts with a combining mark, or violates BiDi rules), punycode is displayed.

There are some limitations such as using [Bidirectional characters](#) (using a domain that has both Left to Right and Right to Left scripts, using a domain that uses combining marks (like ◌̂ which adds a grave on top of the following character) We’ve decided that there are plenty of phishing bypass opportunities, so we don’t need to delve into bypassing these rules and just ignored this policy section by not using combining marks and not violating Bidi Rules.

-

If there is a character in a label not belonging to [Characters allowed in identifiers per Unicode Technical Standard 39 \(UTS 39\)](#), show punycode.

Chrome’s already done most of the work for us and the processed list can be found in chrome’s [code](#).

-

If any character in a label belongs to [the disallowed list](#), show punycode. Chrome's code contains all of the characters that are blacklisted under this section. They can be fetched from the following [file](#) so we've removed them from the list in that last section.

-

If the component uses characters drawn from multiple scripts, it is subject to a script mixing check based on ["Highly Restrictive" profile of UTS 39](#) with an additional restriction on Latin. If the component fails the check, show the component in punycode.

-

Latin, Cyrillic or Greek characters cannot be mixed with each other.

-

Latin characters in the ASCII range can be mixed ONLY with Chinese (Han, Bopomofo), Japanese (Kanji, Katakana, Hiragana), or Korean (Hangul, Hanja).

-

Han (CJK Ideographs) can be mixed with Bopomofo.

-

Han can be mixed with Hiragana and Katakana.

-

Han can be mixed with Korean Hangul.

In order to understand what is script mixing, we need to know a few definitions first: what are the script confusables subclasses and how are they being decided.

So what exactly are Single script / Mixed script / Whole script confusables?

As mentioned in the [Unicode technical standard #39](#):

*"The definitions of these three classes of confusables depend on the definitions of *resolved script set* and *single-script*..."*

Down the rabbit hole we continue to go...

A resolve script set for a single string is an intersection of all of the scripts that the characters of the string belongs to.

An augmented resolve set is a modification to this definition, in which entries for the writing systems containing multiple scripts — Hanb (Han with Bopomofo), Jpan (Japanese), and Kore (Korean) are added, according to the following rules:

-

If Script_Extensions contains Hani (Han), add Hanb, Jpan, and Kore.

-

If Script_Extensions contains Hira (Hiragana), add Jpan.

-

If Script_Extensions contains Kana (Katakana), add Jpan.

-

If Script_Extensions contains Hang (Hangul), add Kore.

-

If Script_Extensions contains Bopo (Bopomofo), add Hanb.

Sets containing Zyyy (Common) or Zinh (Inherited) are treated as ALL, the set of all script values. This is because when intersecting with other code points that belong to other scripts, the resolved intersected script set will contain the “main” script.

The augmented resolve set is the one used in practice for both deciding the resolve set and whether the script is a single script.

the resolved script set ignores characters with Script_Extensions {Common} and {Inherited} and augments characters with CJK (Chinese, Japanese and Korean) scripts with their respective writing systems. Characters with the Script_Extension property values COMMON or INHERITED are ignored when testing for differences in script. Resolved script set for a string is the intersection of the augmented script sets over all characters in the string. This means that if a string has at least 2 different scripts that are not Common or inherited or follow one of the rules for CJK in section 5.1 of the Unicode standard 39, the intersection will be an empty set – and the script will be regarded as a mix script. Otherwise, the intersection set will be non-empty and the script will be regarded as a single script. It is worth noting the even the Unicode standard mention that this definition is confusing as a single script is defined be a resolve set that contains AT LEAST one script but could easily contain more.

Let’s take an example to better understand this:

[image: image]

Circle	U+0421	{Cyr}	{Cyr}	{Cyr}	Yes
	U+0456	{Cyr}	{Cyr}		
	U+0433	{Cyr}	{Cyr}		
	U+0441	{Cyr}	{Cyr}		
	U+04C0	{Cyr}	{Cyr}		
	U+0435	{Cyr}	{Cyr}		
Circle	U+0421	{Cyr}	{Cyr}	∅	No
	U+0069	{Latn}	{Latn}		
	U+0072	{Latn}	{Latn}		
	U+0441	{Cyr}	{Cyr}		
	U+006C	{Latn}	{Latn}		
	U+0435	{Cyr}	{Cyr}		
Circle	U+0043	{Latn}	{Latn}	{Latn}	Yes
	U+0069	{Latn}	{Latn}		
	U+0072	{Latn}	{Latn}		
	U+0063	{Latn}	{Latn}		
	U+0031	{Zyyy}	ALL		
	U+0065	{Latn}	{Latn}		

㇏ 切	U+3006 U+5207	{Hani, Hira, Kata} {Hani}	{Hani, Hira, Kata, Hanb, Jpan, Kore} {Hani, Hanb, Jpan, Kore}	{Hani, Hanb, Jpan, Kore}	Yes
ねガ	U+306D U+30AC	{Hira} {Kata}	{Hira, Jpan} {Kata, Jpan}	{Jpan}	Yes

In the 1st example, all of the codepoints are from a single script and thus, the resolved script set is Cyrillic.

In the 2nd example, the string is composed of 2 different scripts that do not fall for any of the augmented resolve set categories and so the string intersection is empty (the intersection between {Cyrillic} and {Latin} is an empty set).

In the 3rd example, all of the code points but one belongs to the Latin script. the remaining codepoint belongs to the Common script which is considered to be under ALL the scripts. This means the intersection will contain the single script “Latin”.

In the 4th example, we see an example of a CJK codepoint (Chinese, Japanese, Korean) that belongs to 3 scripts, Hani, Hira and Kana (Kata) and another codepoint that belongs to the Hani script. When applying the augmented resolve set rules, we get 2 sets that when intersected, gives us a non empty resolve script set.

The 5th example shows that even scripts that seems to be from different script extensions, can resolve to a non empty set thanks to the augmented resolve set rules.

And now we can go back to the definition of a Single Script. Even the Unicode standard admitted that this name is poorly chosen. A single script really means a non empty resolve set of all of a code points in a string and not just a “single script” as common logic would dictate.

Confusables are divided into three classes: single-script confusables, mixed-script confusables, and whole-script confusables.

Single-script confusables are characters that belong to the same script and that were deemed to be confusable.

Single-script confusable strings are 2 strings that are composed entirely of the same script (up to the augmented resolved set rules), with a non-empty intersected resolved set is not empty, and the confusable characters are from the same script. For example: “ljeto” and “ljeto” in Latin (the Croatian word for “summer”), where the first word uses only four codepoints, the first of which is U+01C9 (lj) LATIN SMALL LETTER LJ.

Mixed-script confusables are characters that belong to different scripts, their resolved set is empty, and are deemed confusable.

Mixed-script confusable strings are 2 strings that have some characters from 2 different scripts, with an empty intersected resolved set, and the confusable characters are from different scripts. For example: “paypal” and “paypal”, where the second word has the character U+0430 (a) CYRILLIC SMALL LETTER A.

Whole-script confusables are 2 strings where each of the strings is composed of a single, different script - thus, for each string alone, their intersected resolved set is non-empty,

but if concatenated, their resolved set would be empty. For example: “scope” in Latin and “scope” in Cyrillic.

The “Highly Restrictive” profile of UTS 39, means that the string qualifies as Single Script, or the string is covered under the above restrictions - described at the start of the bullet. This was about it for Firefox (with very few hardcoded additions to it) but Chrome added a lot more restrictions. If you remember, there were the additional restriction on Latin, and on Cyrillic and Greek. The “additional restriction on Latin” for example, means that some extended Latin scripts are not allowed, specifically Latin extension scripts B,C,D and E. So in order for our attack to work we’ll have to work without mixing scripts, and without using blocks like Latin B-E (and other blocks and characters that were removed from the allowed set of Unicode characters that can be found [here](#)).

-

If two or more numbering systems (e.g. European digits + Bengali digits) are mixed, show punycode.

This section is self-explanatory. There are a few numbering systems of which some characters could be confused, and this section aims to prevent that. This, however does not present any real difficulty any more than section 5 as just adding a code point that is a digit of some numbering system that is of different script will cause the string to be identified as a mix script anyway, so we could just use the regular digits from the ascii range and find confusables in one of the other characters.

-

If there are any invisible characters (e.g. a sequence of the same combining mark or a sequence of Kana combining marks), show punycode.

This section refers to a list of “invisible” characters which includes combining marks (like gravity - U+0300). As put in Chrome’s comment: “combining marks attached to non-LGC characters are already blocked”. So we shouldn’t use combining marks at all unless we use the {Latin,Greek,Cyrillic} scripts.

-

If there are any characters used in an unusual way, show punycode. E.g. [LATIN MIDDLE DOT \(·\)](#) used outside [ela geminada](#).

One approach would be to use these characters only in their context, but this is a very specific limitation. It was easier to bypass it by avoiding it altogether and not using these characters at all. It seems that the functions that holds all the relevant characters are about [here](#).

-

Test the label for [mixed script confusable per UTS 39](#). If mixed script confusable is detected, show punycode.

This section adds on the previous sections (especially 5) as section 5 referred to just the existence of script mixing whereas this section discusses confusables in mix script strings. For testing for mix script confusables, we convert the string to NFD format. What exactly is

NFD (Normalization Form D) format? It is fully explained in Unicode Standard Annex #15. To summarize, NFD is a Canonical Decomposition in which the normalized text is canonical equivalent to the original unnormalized text. This decomposed text is easier to compare in order to see if confusables are present in the current string.

In browsers this is done through the use of the ICU package with the SpoofChecker class that uses the getSkeleton function which used NFD to compare between characters. Then, for each script found in the given string, we see if all the characters in the string outside of that script have whole-script confusables for that script. Because of section 5, we don't really have to worry about multiple scripts in a single string. Of course there are CJK scripts and there are "Common" and "Inherited" scripts but as the majority of focus had been placed on confusables for the ASCII characters, for this research we assumed that the string is a single script and thus we can redact the check to be for the "main" script characters for whole script confusable. In order to bypass that, we just need one character in that substring that is of the same script and is not confusable with the character we want to spoof so the whole string will be shown as Unicode (up to this section)

-

*Test the label for [whole script confusables](#): If all the letters in a given label belong to a set of whole-script-confusable letters in one of the [whole-script-confusable scripts](#) and if the hostname doesn't have a corresponding [allowed top-level-domain](#) for that script, show punycode. **Example for Cyrillic:** The first label in hostname `apple.com (xn--80ak6aa92e.com)` is all [Cyrillic letters that look like Latin letters](#) **AND** the TLD (`com`) is not Cyrillic **AND** the TLD is not one of the TLDs known to host a large number of Cyrillic domains (e.g. `ru`, `su`, `pyc`, `ua`). Show it in punycode.*

This was the biggest change in the updated policy. Before that, this section was phrased like this: If a hostname belongs to a non-IDN TLD(top-level-domain) such as 'com', 'net', or 'uk' and all the letters in a given label belong to a set of Cyrillic letters that look like Latin letters (e.g. Cyrillic Small Letter IE - e), show Punycode. This was in itself an add-on to the policy as a result of the research done by Xudong Zheng. But this fix only applied to the Cyrillic script. And it turned out to be a private case of a broader issue. We've found a few scripts that whole script confusable for the Latin characters can be found. This includes among many, Cyrillic, Greek, Georgian and Armenian. In its new policy, Chrome actually regards this in a function called [WholeScriptConfusableData](#). We reported it to Google and apparently they were already aware of this and tried to fix some of the issues previously reported to them and might added some of our findings. However, as we can see in the diagram, even after reporting and multiple releases, not all of the fixes had been applied.

-

If the label contains only [digits and digit spoofs](#), show punycode.

As this is an edge case when the domain is only composed of digits, we ignored it for this research as most domains and more of all, most practical domains aren't composed of just digits.

-

The site “ßooking.com” is marked in orange as the browsers just transforms it into “ssooking.com” which might fool very limited number of users that looks briefly for “xn-”

or very major change in how the domain looks like in the address bar.

The green colored domains are the domains that are being displayed in Punycode.

In the list above we can see that between Chrome 78 (about when the original report was sent to Google) and now we have a significant improvement in the way The chromium based browsers detect faked sites. Firefox and Safari behavior did not change between version Firefox 70.0 and Safari 13.0.2 (15608.2.30.1.1)

Although the old Explorer/Edge browsers seems to be more “Secure” in the way they display sites that have code points that are not included in the languages installed on the user’s OS, another phishing attack may take place, in which an attacker will register on purpose a site that displays in Punycode when the original site is also being displayed in Punycode to a big enough group of users (or just a targeted users who uses it) but will change a Unicode that its codepoint will display as almost the same as the original Unicode character. For example, lets take xn-oncinema-i0a.com and xn-oncinema-ioa.com The 1st will translate to æoncinema.com in Unicode and the 2nd to onc·inema.com

In domain line #9 the domain ωωε.com is somewhat similar to the domain wwe.com and eḃay.com is similar to ebay.com As we can see, although implemented and improved the code regarding whole-script confusable, there’s still room for improvement.

Many of the examples above have a confusable Latin character like rottentomatœs.com This is no coincidence, as the Latin script aggregates many languages under its wing even without adding the Latin extension scripts (B-E). Chrome has done a good job trying to root out these confusables as can be seen from the difference from the older Chrome 78 (which in itself fixed a few more from earlier versions)

□ifewire.com is using a Japanese letter (Specifically Hiragana Letter Si) with Latin Letters. This is allowed as CJK languages has exceptions in the policy that allows them to use Latin characters in domain names.

At lines #24-31 we can see that the naive approach Safari’s developers chose, which is to allow certain types of scripts but not others, exposing their users to phishing attempts. Lines #30-31 takes this to an absurd level (for the demonstration) where in Line #30 a Hebrew punctuation mark is being displayed as an “o” in the (quite obviously) fake booking.com In line #31 we can see scripts that are all allowed by Safari mixed together to a Gibberish outcome but Safari still shows them in their Unicode form.

Line #34 was added just as an example. We can see a domain that seems like EBAY.com spelled in capital letters where actually it’s Greek capital letters. No browser displays domain names in capital letters, so the way Chromium based browsers and Firefox chose to display it was in Unicode form, as εβayu.com doesn’t seems like ebay.com (and the fact that β is not treated as confusable with b but that’s another story) Moreover, it shows that browsers compare only lowercase letters (which they should).

Now, the main methodology that we used for finding confusable domains, was to find codepoints that looks like known ASCII alphanumeric code points but that do not decompose to ASCII alphanumeric code points as otherwise, it would already be used in the ICU library to indicate a confusable, or used in browsers that can decompose code

point and compare the decomposed string to a known domain list (like section 13 in Chrome's IDN police).

In order to do that, we developed a tool that goes over the Unicode code points, and creates a list with attributes for each code point: Script, skeleton, description, Case.

then we supply a domain name that we want to fake and filter code points according to some restrictions that we can set to bypass certain sections in the policy. For example, if we wanted to create a fake "Google" domain we roughly:

- Choose a script (like Latin)
- Filter all of the characters that are confusable to other characters (in the assumption that they are not similar to other characters)
- Might use further filter to remove all the code points that has "G" character in their description that describe a similarity to G (not just G in any of the string but looks for "Letter G")
- Filter more characters that got into the disallowed blacklist of Chrome.
- For domain that are in the [domains.list](#), filter all of the code points that can be decomposed through NFD to ASCII code points

The result is a list that contains all of the remaining code points for each character in the string, from which we chose a single character that looks like one of the domain's characters.

Remember, in order to create a fake domain, we need only to find one code point that will not resolve to an ASCII code point and is not in some blacklist. This is because of the fact that when one code point cannot be resolved to an ASCII code point, the comparison of the whole string to an existing domain will not succeed.

So we pick one character from the list and compose the rest of the string with any of the characters that we can use from that script - we don't care if they're confusable or not.

That's about it for this section. We also investigated Unicode in IM, but that will be a topic for the next blogpost (which will be much shorter).

Future research

The following research avenues have come up during the research:

-

Any mechanism that relies on comparing strings can be vulnerable to this attack if there exist another mechanism that transforms the compared string in any way before comparison like Authentication, Authorization, Automated help center and so on.

*As we were thinking to explore this, John Gracey published a [post](#) that discuss Github's authentication bypass using characters like dotless i and other dangerous patterns.

- On the social engineering side, human help center might be fooled as well.
- Polyglots shell commands or code might be an interesting avenue to explore.

Acknowledgments

The research was inspired originally by the work of [Xudong Zheng](#).

And while reviewing the literature on this topic we've found a few other papers:

Adrian Crenshaw - [Out of Character: Use of Punycode and Homoglyph Attacks to Obfuscate URLs for Phishing](#)

The Tarquin - [Weaponizing Unicode Homographs Beyond IDNs](#)

And later in the research process we've discovered:

Deep Confusables - [Improving Unicode Encoding Attacks with Deep Learning](#) by Miguel Hernández (G@MiguelHzBz), José Ignacio Escribano (I@jiep) and Dr. Alfonso Muñoz (G@mindcrypt)

Archived from <https://alephsecurity.com/2020/07/23/revised-homograph-attacks2/>. Rendered offline from the local Markdown copy.